COMP 3311
DATABASE MANAGEMENT SYSTEMS

LECTURE 9
STRUCTURED QUERY LANGUAGE (SQL): DATA MANIPULATION LANGUAGE (DML)

AGGREGATE FUNCTIONS

- An aggregate function operates on a relation attribute and returns a relation with one row and one column (i.e., a single value).

count	number of tuples / values	**avg**	average value
stdev	standard deviation of values	**max**	maximum value
sum	sum of values (total)	**min**	minimum value

- For **avg**, **stdev** and **sum** the input must be numbers.
- For other aggregate functions, the input can be non-numeric (e.g., strings).
- All aggregate functions, except **count**(*), ignore null values in the input collection and return a value of null for an empty collection.

☞ **The count of an empty (null) collection is defined to be 0.**

EXAMPLE AGGREGATE FUNCTIONS

Query: Find the minimum, maximum and average credit rating of clients.

```
select min(rating) as minRating,
       max(rating) as maxRating,
       avg(rating) as avgRating
from Client;
```

Query result

minRating	maxRating	avgRating
6	9	7.25

Query: Find the average, median and total balance of accounts.

```
select avg(balance) as average,
       median(balance) as median,
       sum(balance) as total
from Account;
```

Query result

average	median	total
58000	55000	290000

AGGREGATE FUNCTIONS: COMPUTATION

Query: Find the average account balance at the Pacific Place branch.

```
select avg(balance) as avgBalance
from Account
where branchName='Pacific Place';
```

First, select the tuples from the Account relation where branch name equals Pacific Place.

Then, project on the balance attribute retaining duplicates.

Finally, calculate the average.

Query result

avgBalance
60000

Account

accountNo	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

accountNo	balance	branchName
A-301	75000	Pacific Place
A-322	45000	Pacific Place

balance
75000
45000

AGGREGATE FUNCTIONS: EXAMPLES

Query: Find the number of accounts.

```
select count(*)
from Account;
```

Query result

count(*)
5

☞ **Remember * stands for *all* attributes**

⇒ count all tuples.

Same as:

```
select count(branchName)
from Account;
```

Query result

count(branchName)
5

Why?

Different from:

```
select count(distinct branchName)
from Account;
```

Query result

count(distinct branchName)
3

Why?

Cannot say:

```
select count(distinct *)
from Account;
```

SQL does not allow the use of **distinct** with **count**(*).

GROUP BY CLAUSE

A **group by** clause permits aggregate results (e.g., **max**, **min**, **sum**, etc.) to be displayed for groups.

For example, **group by** x will get a result for every different value of x.

☞ **Aggregate queries without group by return a single value.**

☞ **Aggregate queries with group by return one value for each group.**

Query: Find the number of accounts at *each* branch.

```
select branchName, count(*)
from Account
group by branchName;
```

Query result

branchName	count(*)
Star House	2
Diamond Hill	1
Pacific Place	2

GROUP BY CLAUSE (CONT'D)

Query: Find the number of accounts at *each* branch.

```
select branchName, count(*)
from Account
group by branchName;
```

First, group the tuples in the Account relation by branch name.

Then, for each group, count the number of tuples in the group.

Account

accountNo	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

accountNo	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

Query result

branchName	count(*)
Star House	2
Diamond Hill	1
Pacific Place	2

GROUP BY CLAUSE: REQUIRED ATTRIBUTES

Query: Find the balance and number of accounts at *each* branch.

```
select branchName, balance, count(*)
from Account
group by branchName;
```

Illegal SQL! Why?

Query result

accountNo	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

A non-aggregated attribute in the select clause must also appear in the group by clause.
(This ensures that there is only one value of the attribute for each group.)

The opposite is not true!

Attributes in the group by clause do not need to appear in the select clause.
(Can group on attributes not in the **select** clause.)

SQL Note
Some relational systems allow this type of query. However, the query result is non-deterministic.

GROUP BY CLAUSE: REQUIRED ATTRIBUTES (CONT'D)

Query: Find the balance and number of accounts at *each* branch.

```
select branchName, balance, count(*)
from Account
group by branchName, balance;
```

Query result

branchName	balance	count(*)
Star House	35000	1
Star House	80000	1
Diamond Hill	55000	1
Pacific Place	75000	1
Pacific Place	45000	1

Query: Find the total balance and number of accounts at *each* branch.

```
select branchName, sum(balance), count(*)
from Account
group by branchName;
```

Query result

branchName	sum(balance)	count(*)
Star House	115000	2
Diamond Hill	55000	1
Pacific Place	120000	2

Both queries are legal **SQL** since for each query result tuple there is only a single balance value.

GROUP BY CLAUSE: WITH JOIN

Query: Find the unique number of depositors for each branch.

```
select branchName, count(distinct clientId)
from Depositor natural join Account
group by branchName;
```

JOIN ⇒ (clientId, accountNo, balance, branchName)

clientId	…	branchName
3	…	Star House
3	…	Star House
4	…	Pacific Place
8	…	Star House
3	…	Pacific Place
4	…	Diamond Hill

group by

clientId	…	branchName
3	…	Star House
~~3~~	~~…~~	~~Star House~~
8	…	Star House
4	…	Pacific Place
3	…	Pacific Place
4	…	Diamond Hill

distinct

clientId	…	branchName
3	…	Star House
8	…	Star House
4	…	Pacific Place
3	…	Pacific Place
4	…	Diamond Hill

count

Query result

branchName	count(clientId)
Star House	2
Pacific Place	2
Diamond Hill	1

☞ **Group by and aggregate functions are applied to a join result.**

HAVING CLAUSE

The **having** clause applies a condition to select the groups to retain.

☞ **The having clause can only be used with the group by clause and, moreover, it requires a group by clause.**

Query: Find the name and average balance of the branches where the average account balance is more than $55000.

```
select branchName, avg(balance)
from Account
group by branchName
having avg(balance)>55000;
```

Any condition that appears in the **having** clause applies to the groups and is evaluated ***after*** the formation of the groups.

Account

	accountNo	balance	branchName	
✓	A-117	35000	Star House	avg(57500)
	A-120	80000	Star House	
X	A-215	55000	Diamond Hill	avg(55000)
	A-301	75000	Pacific Place	
✓	A-322	45000	Pacific Place	avg(60000)

⟹

Query result

branchName	avg(balance)
Star House	57500
Pacific Place	60000

HAVING CLAUSE: EVALUATION

Query: Find the branch names in Yau Tsim Mong district where the average account balance is more than $50000.

evaluation order

```
5. select branchName as branch
1. from Account natural join Branch
2. where district='Yau Tsim Mong'
3. group by branchName
4. having avg(balance)>50000
6. order by branchName;
```

First, find the records returned by the **from** clause that satisfy the **where** clause predicate.

Then, form the groups (only tuples that satisfy the **where** clause predicate are included when forming groups).

Finally, apply the **having** clause to *each group*.

Select clause alias use in SQL queries

In most relational DBMSs, an alias defined in the **select** clause cannot be used in most other clauses. This is because the order of evaluation of an **SQL** query is:

1. **from** clause
2. **where** clause
3. **group by** clause
4. **having** clause
5. **select** clause
6. **order by** clause

Consequently, a column or alias name defined in a clause evaluated later cannot be used in a clause evaluated earlier. Specifically, an alias defined in a **select** clause cannot be used in a **where** clause, **group by** clause or **having** clause.

☞ **The having clause predicate can only qualify on attributes specified in the group by clause or on aggregate values.**

Account(accountNo, balance, *branchName*)

Branch(branchName, district, assets)

HAVING CLAUSE: EVALUATION (CONT'D)

1. Evaluate the **from** clause to get a relation.
2. If a **where** clause is present, apply the predicate in the **where** clause to the **from** clause result relation before the formation of groups.

   ☞ **Records that do not pass the where clause predicate are eliminated *before* the formation of groups.**

3. If a **group by** clause is present, group tuples satisfying the **where** clause predicate into groups. If the **group by** clause is absent, the entire set of tuples satisfying the **where** clause predicate is treated as a group.

   If a **having** clause is present, apply the predicate to each group retaining only those groups satisfying the **having** clause.

   ☞ **Any attribute present in the having clause *that is not being aggregated* must appear in the group by clause.**

4. Evaluate the aggregate functions in the **select** clause to get a single result for each group.

DUPLICATE TUPLES TEST

- The **group by** and **having** clauses can test for the *nonexistence* (absence) and *existence* (presence) of duplicate tuples.

Query: Find the id of clients who have *only one account* at the Star House branch.

Query result

clientId
8

```
select clientId
from Depositor D, Account A
where D.accountNo=A.accountNo
        and branchName='Star House'
group by clientId
having count(*)=1;
```

Query: Find the id of clients who have *at least two accounts* at the Star House branch.

Query result

clientId
3

How would you express this as an SQL query?

NESTED SUBQUERIES

- Every **SQL** statement returns a relation as the result.

  ☞ **A relation can be empty or contain only a single, atomic value.**

- Consequently, a value or a set of values can be replaced with a **SQL** statement (i.e., with a subquery ⟹ nested query).

```
select *
from Loan
where amount>12000;
```

This value can be replaced with a subquery.

```
select *
from Loan
where amount>(select avg(amount)
              from Loan);
```

This subquery *must* return a single, numeric value else the query is illegal.

☞ **The query is illegal if the subquery returns the wrong number of values/tuples or the wrong data type for the comparison.**

Subqueries are commonly used to test for set membership, perform set comparison or determine set cardinality.

EXAMPLE NESTED SUBQUERY

Query: Find the client id, name and credit rating of all clients with the third highest credit rating. Order the result by client name ascending.

```
select clientId, name, rating
from Client
where rating=(select distinct rating
              from Client
              order by rating desc nulls last
              offset 2 rows
              fetch next 1 row only)
order by name;
```

Select the clients whose rating is the third highest.

Get the third highest rating.

Query result

clientId	name	rating
8	Isaac Li	6
4	Mary Kwan	6

To return all clients with the third highest credit rating, it is necessary to first use a subquery to get the third highest rating.

Then, the clients who have this rating can be retrieved.

SET MEMBERSHIP: IN

Query: Find the id of clients who have both an account and a loan.

```
select distinct clientId
from Borrower
where clientId in (select clientId
                   from Depositor);
```

First, find the set of clients who have an account (i.e., who are depositors).

Then, check if the clients who have a loan (i.e., who are borrowers) are in the set of clients who have an account (i.e., who are depositors).

Query result

clientId
3
4

The **in** operator tests for the presence of set membership (i.e., selects clients in the Borrower set *only if* they are in the Depositor set). ***Duplicates are retained.***

clientId
3
4
17

Borrower set

The *set* of clients who have an account.

clientId
3
4
8

Depositor set

☞ **The subquery must return values of the same data type as the comparison attribute.**

SET MEMBERSHIP: NOT IN

Query: Find the id of clients who have a loan, but do not have an account.

```
select distinct clientId
from Borrower
where clientId not in (select clientId
                       from Depositor);
```

First, find the set of clients who have an account (i.e., who are depositors).

Then, check if the clients who have a loan (i.e., who are borrowers) are not in the set of clients who have an account (i.e., who are depositors).

Query result

clientId
17

The **not in** operator tests for the absence of set membership (i.e., selects clients in the Borrower set *only if* they are not in the Depositor set). ***Duplicates are retained.***

clientId
3
4
17

Borrower set

The *set* of clients who have an account.

clientId
3
4
8

Depositor set

SET TO SET COMPARISON

Query: Find, for each branch, the branch name, client id, client name, client account number and client account balance for the clients with the largest account balance.

```
select branchName, clientId, name, accountNo, balance
from Client natural join Depositor natural join Account
where (branchName, balance) in (select branchName, max(balance)
                                from Account
                                group by branchName);
```

Note
The sets must have the same types of values in the same order.

Account

accountNo	balance	branchName
A-117	35000	Star House
A-120	80000	Star House
A-215	55000	Diamond Hill
A-301	75000	Pacific Place
A-322	45000	Pacific Place

branchName	max(balance)
Star House	80000
Diamond Hill	55000
Pacific Place	75000

The *set* of branches and their maximum balances.

Query result

branchName	clientId	name	accountNo	balance
Star House	3	Steven Chan	A-120	80000
Star House	8	Isaac Li	A-120	80000
Diamond Hill	4	Mary Kwan	A-215	55000
Pacific Place	3	Steven Chan	A-301	75000

Depositor(*clientId, accountNo*)

Account(accountNo, balance, *branchName*)

Client(clientId, name, hkid, address, district)

☞ **Sets of values can be compared.**

SET COMPARISON: SOME

Query: Find the name and assets of the branches that have greater assets than *some* (i.e., at least one) branch located in Yau Tsim Mong district.

```
select branchName, assets
from Branch
where assets >some (select assets
                    from Branch
                    where district='Yau Tsim Mong');
```

Query result

branchName	assets
Star House	115000

The **where** clause is true if the assets value of a Branch tuple is greater than at least one member of the set of all assets values of branches in Yau Tsim Mong (i.e., greater than the minimum assets value in the set). ***Duplicates are retained.***

The *set* of assets values of all branches in Yau Tsim Mong.

assets
107000
115000

SET COMPARISON: SOME

Query: Find the name and assets of the branches that have greater assets than *some* (i.e., at least one) branch located in Yau Tsim Mong district.

```
select branchName, assets
from Branch
where assets >some (select assets
                    from Branch
                    where district='Yau Tsim Mong');
```

☞ Equivalent to: Find the name and assets of the branches that have greater assets than the minimum assets of any branch located in Yau Tsim Mong district.

```
select branchName, assets
from Branch
where assets > (select min(assets)
                from Branch
                where district='Yau Tsim Mong');
```

SET COMPARISON: SEMANTICS OF SOME

(5 < some | 0 | 5 | 6 |) returns true (since 5 is less than the maximum value 6 in the set)

(5 < some | 0 | 5 |) returns false (since 5 is not less than the maximum value 5 in the set)

(5 = some | 0 | 5 |) returns true (since 5 = 5)

(5 ≠ some | 0 | 5 |) returns true (since 5 ≠ 0)

SQL Notes

1. = **some** is equivalent to **in**.
2. ≠ **some** is not equivalent to **not in**.

SET COMPARISON: ALL

Query: Find the name and assets of the branches that have greater assets than *all* branches located in Central and Western district.

```
select branchName, assets
from Branch
where assets >all (select assets
                   from Branch
                   where district='Central and Western');
```

Query result

branchName	assets
Festival Walk	107000
Star House	115000

The **where** clause is true if the assets value of a Branch tuple is greater than each of the members of the set of all assets values of branches in Central and Western (i.e., greater than the maximum assets value in the set). ***Duplicates are retained.***

The *set* of assets values of all branches in Central and Western.

assets
40000

SET COMPARISON: ALL

Query: Find the name and assets of the branches that have greater assets than *all* branches located in Central and Western district.

```
select branchName, assets
from Branch
where assets >all (select assets
                   from Branch
                   where district=' Central and Western');
```

☞ Equivalent to: Find the name and assets of all branches that have greater assets than the maximum assets of any branch located in Central and Western district.

```
select branchName, assets
from Branch
where assets > (select max(assets)
                from Branch
                where district=' Central and Western');
```

SET COMPARISON: SEMANTICS OF ALL

(5 < **all** | 0 / 5 / 6 |) returns false (since 5 is not less than all the members in the set)

(5 < **all** | 6 / 9 |) returns true (since 5 is less than all the members in the set)

(5 = **all** | 4 / 5 |) returns false (since 5 ≠ 4)

(5 ≠ **all** | 6 / 9 |) returns true (since 5 ≠ 6 or 9)

SQL Notes

1. ≠ **all** is equivalent to **not in**.
2. = **all** is not equivalent to **in**.

EMPTY RELATION TEST

- The **exists** operator returns true if the subquery is *not empty* (i.e., the subquery returns at least one tuple).

Query: Find the id of clients who have both an account and a loan.

If the subquery result is not empty, return the client id.

```
select clientId
from Depositor D
where exists (select *
              from Borrower B
              where D.clientId=B.clientId)
```

Clients who have both a loan and an account are in the subquery result.

Query result

clientId
3
4

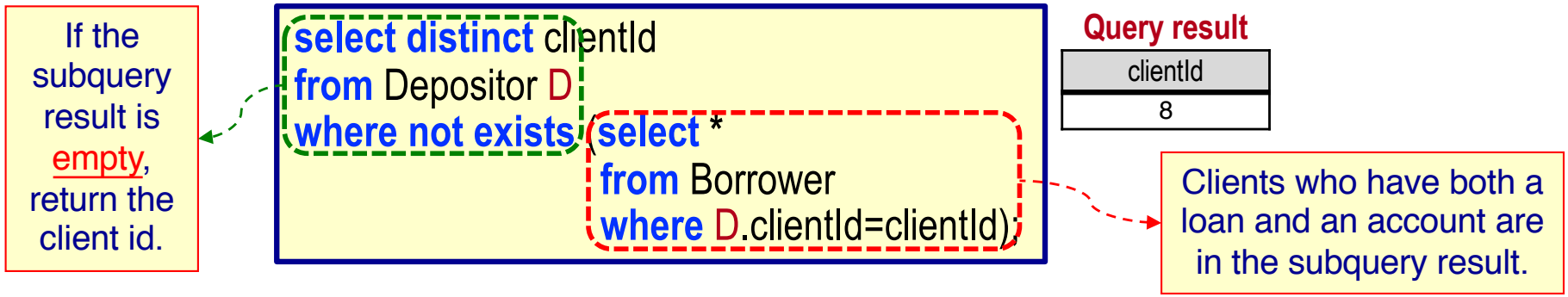
Scoping rules for correlation names (aliases) in subqueries.

- A correlation name defined in a subquery can be used only in the subquery itself or in any subquery contained in the subquery (e.g., D can be used in the subquery (nested **select**); B cannot be used in the outer **select**).
- Locally defined correlation names override globally defined names.

EMPTY RELATION TEST (CONT'D)

- The **not exists** operator returns true if the subquery is *empty* (i.e., the subquery returns no rows).

Query: Find the id of clients who have an account, but do not have a loan.



☞ The **not exists** operator can be used to simulate set containment.

relation A contains relation B ⇔ **not exists** (B **minus** A)

SUBQUERIES IN THE FROM CLAUSE

- The **from** clause can contain a subquery.

Why?

Query: Find the name and average balance of the branches whose average balance is greater than the average account balance.

```
select branchName, avgBalance
from (select branchName, avg(balance) as avgBalance
      from Account
      group by branchName) result
where avgBalance>(select avg(balance)
                  from Account);
```

result

branchName	avgBalance
Pacific Place	60000
Star House	57500
Diamond Hill	55000

The result relation contains the branch name and average balance of each branch.

Query result

branchName	avgBalance
Pacific Place	60000

avg(balance)
58000

The average balance of all accounts.

☞ **The relation result is called a *derived (temporary) relation*.**

(It is created solely for use in the query and is discarded after the query executes.)

Account(accountNo, balance, *branchName*)

SUBQUERIES IN THE FROM CLAUSE (CONTD)

Query: Find the name and average balance of branches with the *maximum average* account balance.

```
select branchName, avgBalance
from (select branchName, avg(balance) as avgBalance
      from Account
      group by branchName) result
where avgBalance=(select max(avgBalance)
                  from result);
```

The maximum average balance in the result relation.

result

branchName	avgBalance
Pacific Place	60000
Star House	57500
Diamond Hill	55000

The result relation contains the branch name and average balance of each branch.

Oracle Note

This query is *not allowed in Oracle* due to Oracle's scoping rules.
(The scope of the result relation is restricted to the outer select clause.)
See the next slide for an alternate way to express this query.

WITH CLAUSE

A **with** clause allows a derived (temporary) relation to be defined that is available only to the query in which the **with** clause occurs.

Query: Find the name and average balance of branches with the *maximum average* account balance.

```
with result (branchName, avgBalance) as
    (select branchName, avg(balance)
     from Account
     group by branchName)
select branchName, avgBalance
from result
where avgBalance=(select max(avgBalance)
                  from result);
```

result

branchName	avgBalance
Pacific Place	60000
Star House	57500
Diamond Hill	55000

The result relation contains the branch name and average balance of each branch.

Query result

branchName	avgBalance
Pacific Place	60000

max(avgBalance)
60000

The maximum average balance in the result relation.

Oracle Note
This query *is allowed* in Oracle.